

# CSE721: Project Encryption/Decryption Tool Report

Submitted By:

Rafiad Sadat Shahir (20101580)

Zayed Humayun (20141030)

Date of submission:

03/01/2026

## Classic Cipher Client

The Classic Cipher Client is a command-line cryptography tool built in Python that provides a unified interface for multiple classical ciphers Caesar, Affine, Playfair, and Hill. The interface uses Python's argparse module, allowing users to perform encryption, decryption, and (for Hill) key recovery directly from the terminal in a consistent and intuitive way.

### Overall Program Help Screen

```
● (base) PS C:\Users\USER\Desktop\721\Project_Crypto-main> python main.py --h
usage: main.py [-h] {caesar,affine,playfair,hill} ...

Classic cipher CLI

positional arguments:
  {caesar,affine,playfair,hill}
  caesar                  Caesar cipher
  affine                  Affine cipher
  playfair                Playfair cipher
  hill                    Hill cipher (2x2)

options:
  -h, --help              show this help message and exit
```

The top-level help screen (python main.py -h) displays the Classic cipher CLI usage, showing the four supported ciphers as positional arguments: caesar, affine, playfair, and hill.

Each leads to subcommands for enc (encryption) or dec (decryption) with cipher-specific keys.

### Hill Cipher Help Screen

```
● (base) PS C:\Users\USER\Desktop\721\Project_Crypto-main> python main.py hill -h
usage: main.py hill [-h] {enc,dec,crack} ...

positional arguments:
  {enc,dec,crack}
  enc              Encrypt with Hill
  dec              Decrypt with Hill
  crack            Crack Hill 2x2 key from known plaintext/ciphertext

options:
  -h, --help              show this help message and exit
```

The Hill subcommand help screen (python main.py hill -h) details the three available modes as positional arguments: enc (encrypt with Hill), dec (decrypt with Hill), and crack (recover 2x2 Hill key from known plaintext/ciphertext).

## Core Architecture

The project centers on four cipher classes CaesarCipher, AffineCipher, PlayfairCipher, and HillCipher each providing a uniform encrypt(plain\_text) and decrypt(cipher\_text) interface for seamless interchangeability. The HillCipher additionally includes crack\_key(plain\_text, cipher\_text) for key recovery from known plaintext-ciphertext pairs.

## Cipher-Specific Details

1. Caesar Cipher: Requires a single integer key (shift value).
2. Affine Cipher: Requires two integer keys — a multiplicative and an additive parameter.
3. Playfair Cipher: Uses a keyword string as the key.
4. Hill Cipher: Accepts a 2×2 matrix key provided as four integers.

## Unified CLI Interface

All cipher operations follow the same general command pattern:

python main.py <cipher> <mode> [options]

- <cipher>: The algorithm to use (caesar, affine, playfair, or hill).
- <mode>: The operation mode (enc for encryption, dec for decryption).
- [options]: Cipher-specific parameters such as keys, input files, and output files.

This uniform interface simplifies testing and usage since the same command structure applies to all ciphers. Input text can be read from a file, and the resulting output can be both displayed in the terminal and saved to a file via command-line arguments.

## Running the Cipher Client

The project provides a unified command-line interface where the user selects the cipher (caesar, affine, playfair, hill), selects the mode (enc for encryption, dec for decryption), provides the required key, and specifies input/output files.

```
• (base) PS C:\Users\USER\Desktop\721\Project_Crypto-main> python main.py caesar enc -k 3 -f texts/plain.txt -o texts/cipher_caesar.txt
CAESAR encrypt | key=3 | input=texts/plain.txt | output=texts/cipher_caesar.txt
WKLV LV WKH SODQ WHAW.
• (base) PS C:\Users\USER\Desktop\721\Project_Crypto-main> python main.py caesar dec -k 3 -f texts/cipher_caesar.txt -o texts/plain_caesar.txt
CAESAR decrypt | key=3 | input=texts/cipher_caesar.txt | output=texts/plain_caesar.txt
this is the plain text.
```

**Caesar encryption:** python main.py caesar enc -k 3 -f texts/plain.txt -o texts/cipher\_caesar.txt

**Caesar decryption:** python main.py caesar dec -k 3 -f texts/cipher\_caesar.txt -o texts/plain\_caesar.txt

```

• (base) PS C:\Users\USER\Desktop\721\Project_Crypto-main> python main.py affine enc --a 5 --b 8 -f texts/plain.txt -o texts/cipher_affine.txt
AFFINE encrypt | key=(a=5, b=8) | input=texts/plain.txt | output=texts/cipher_affine.txt
ZRWU WU ZRC FLIMV ZCTZ.
• (base) PS C:\Users\USER\Desktop\721\Project_Crypto-main> python main.py affine dec --a 5 --b 8 -f texts/cipher_affine.txt -o texts/plain_affine.txt
AFFINE decrypt | key=(a=5, b=8) | input=texts/cipher_affine.txt | output=texts/plain_affine.txt
this is the plain text.

```

**Affine encryption:** python main.py affine enc --a 5 --b 8 -f texts/plain.txt -o texts/cipher\_affine.txt

**Affine decryption:** python main.py affine dec --a 5 --b 8 -f texts/cipher\_affine.txt -o texts/plain\_affine.txt

```

• (base) PS C:\Users\USER\Desktop\721\Project_Crypto-main> python main.py playfair enc -k "keyword" -f texts/plain.txt -o texts/cipher_playfair.txt
PLAYFAIR encrypt | key='keyword' | input=texts/plain.txt | output=texts/cipher_playfair.txt
VFLQ LQ VFY NHGGQ UKVZ.
• (base) PS C:\Users\USER\Desktop\721\Project_Crypto-main> python main.py playfair dec -k "keyword" -f texts/cipher_playfair.txt -o texts/plain_playfair.txt
PLAYFAIR decrypt | key='keyword' | input=texts/cipher_playfair.txt | output=texts/plain_playfair.txt
this is the plain text.

```

**Playfair encryption:** python main.py playfair enc -k "keyword" -f texts/plain.txt -o texts/cipher\_playfair.txt

**Playfair decryption:** python main.py playfair dec -k "keyword" -f texts/cipher\_playfair.txt -o texts/plain\_playfair.txt

```

• (base) PS C:\Users\USER\Desktop\721\Project_Crypto-main> python main.py hill enc --key 3 2 8 5 -f texts/plain.txt -o texts/cipher_hill.txt
HILL encrypt | key=[[3, 2], [8, 5]] | input=texts/plain.txt | output=texts/cipher_hill.txt
JVMC MC JVC FHWYD LGNL.
• (base) PS C:\Users\USER\Desktop\721\Project_Crypto-main> python main.py hill dec --key 3 2 8 5 -f texts/cipher_hill.txt -o texts/plain_hill.txt
HILL decrypt | key=[[3, 2], [8, 5]] | input=texts/cipher_hill.txt | output=texts/plain_hill.txt
this is the plain text.

```

**Hill encryption:** python main.py hill enc --key 3 2 8 5 -f texts/plain.txt -o texts/cipher\_hill.txt

**Hill decryption:** python main.py hill dec --key 3 2 8 5 -f texts/cipher\_hill.txt -o texts/plain\_hill.txt

```

• (base) PS C:\Users\USER\Desktop\721\Project_Crypto-main> python main.py hill crack --plain-file texts/plain.txt --ciphertext texts/cipher_hill.txt -o keys/hill_cracked.txt
3 2
8 5

```

The **crack\_key** method in HillCipher performs a known-plaintext attack to recover the 2x2 Hill key matrix. It aligns cleaned plaintext and ciphertext, builds small 2x2 blocks, and finds one whose plaintext block is invertible modulo 26 so it can compute  $\text{key} = P^{-1} \cdot C$  and return that matrix as the cracked key.

## Input Validation and Error Handling

```

• (base) PS C:\Users\USER\Desktop\721\Project_Crypto-main> python main.py hill enc --key 1 2 3 4 -f texts/plain.txt -o texts/cipher_hill.txt
Error: Invalid Hill key: det(K) mod 26 must be coprime with 26. Got det mod 26 = 24.

```

Additionally, input validation is included (especially for Hill) so that invalid keys (e.g., non-invertible matrices under mod 26) are detected early and reported with a clear error message instead of causing a crash.

## Project Structure

Project\_Crypto/

```
|— keys/      # Storage for generated keys
|— src/       # Python modules implementing cryptographic functions
|— texts/     # Input and output text files
|— main.py    # Main script for encryption/decryption workflows
|— README.md
```

Key components include `main.py` as the entry point, `src/` containing all cipher implementations, `keys/` for key storage, and `texts/` for input/output files.

## Requirements

- Python 3.7 or higher
- No additional packages required (uses only standard library)

## Standard Libraries used

The CLI uses Python's `argparse` library to parse command-line arguments and implement subcommands for each cipher (caesar, affine, playfair, hill).

<https://docs.python.org/3/library/argparse.html>

The Hill and Affine implementations use Python's built-in `pow(base, exp, mod)` for modular arithmetic; in particular, `pow(x, -1, 26)` computes a modular inverse when it exists.

<https://docs.python.org/3/library/functions.html#pow>

The tool also uses the standard math library (`math.gcd`) to validate keys (for example, checking if a value is invertible modulo 26).

<https://docs.python.org/3/library/math.html>

## Github Link

[https://github.com/DarkerNemesis14/Project\\_Crypto](https://github.com/DarkerNemesis14/Project_Crypto)